PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA

Conceitos iniciais de POO

Nesta aula serão apresentados os fundamentos necessários para compreender a organização de programas Java por meio de classes e objetos, estabelecendo a base para conteúdos posteriores como encapsulamento, herança, polimorfismo, MVC e DAO.

Fundamento

Classe

Modelo utilizado para definir dados e comportamentos de uma entidade.

Execução

Objeto

Instância concreta criada a partir da estrutura definida por uma classe.

Comportamento

Método

Ação que um objeto pode executar dentro do sistema.

1. INTRODUÇÃO

O que é Programação Orientada a Objetos?

A Programação Orientada a Objetos, ou **POO**, é um paradigma de desenvolvimento no qual o sistema é organizado a partir de estruturas que representam entidades, conceitos e responsabilidades.

> **Ideia central:**
>
> Em vez de concentrar todo o código em uma sequência extensa de comandos, os dados e comportamentos relacionados são agrupados em objetos.

Sistema acadêmico

Aluno, Professor, Disciplina, Turma, Matrícula e Nota podem ser representados por classes diferentes.

Sistema comercial

Cliente, Produto, Pedido, Pagamento e Estoque também podem ser representados por objetos.

2. ORGANIZAÇÃO DO SOFTWARE

Por que utilizar POO?

Organização

As responsabilidades do sistema podem ser separadas em classes, reduzindo a concentração de código em um único ponto.

Manutenção

Alterações tendem a ficar restritas às classes responsáveis pelo comportamento modificado.

Reutilização

Uma classe bem projetada pode ser utilizada diversas vezes e em diferentes partes da aplicação.

Modelagem

Conceitos do problema podem ser traduzidos para estruturas de software mais fáceis de compreender.

3. MUDANÇA DE ABORDAGEM

Programação procedural × POO

Abordagem procedural

O foco principal está na sequência de operações e em funções que manipulam dados.

```
cadastrarAluno();
calcularMedia();
mostrarAluno();
```

Orientação a objetos

Os comportamentos são associados ao objeto responsável por eles.

```
Aluno aluno = new Aluno();

aluno.calcularMedia();
aluno.mostrarDados();
```

4. VOCABULÁRIO ESSENCIAL

Conceitos fundamentais

Classe → **Objeto** → **Atributos** + **Métodos**

Classe

Define a estrutura comum de um determinado tipo de objeto.

Objeto

É uma instância criada a partir de uma classe.

Atributo

Representa os dados ou características do objeto.

Método

Representa ações ou comportamentos executados pelo objeto.

5. CLASSES

Classe: o modelo do objeto

Uma classe descreve quais informações e comportamentos estarão disponíveis para os objetos criados a partir dela.

```
public class Pessoa {

    String nome;
    int idade;

}
```

> Neste momento existe apenas a definição de como uma **Pessoa** será representada. Ainda não foi criada uma pessoa específica.

6. OBJETOS E INSTÂNCIAS

Objeto: uma ocorrência concreta

Um objeto é criado a partir de uma classe. Diversos objetos podem ser gerados utilizando o mesmo modelo.

```
Pessoa pessoa1 = new Pessoa();
Pessoa pessoa2 = new Pessoa();

pessoa1.nome = "Ana";
pessoa1.idade = 20;

pessoa2.nome = "Carlos";
pessoa2.idade = 35;
```

Os dois objetos pertencem à mesma classe, mas armazenam estados diferentes.

7. COMPARAÇÃO

Classe × Objeto

Classe	Exemplo de objeto
Carro	Um Toyota Corolla específico
Aluno	Uma aluna chamada Maria
Produto	Um notebook cadastrado no estoque
ContaBancaria	A conta de um determinado cliente

> **Classe = modelo. Objeto = elemento criado a partir desse modelo.**

8. CRIAÇÃO DE OBJETOS

Instanciação em Java

```
Pessoa pessoa = new Pessoa();
```

Pessoa

Define o tipo da referência.

pessoa

Nome da variável que fará referência ao objeto.

new

Solicita a criação de uma nova instância.

Pessoa()

Representa a chamada ao construtor da classe.

9. ESTADO

Atributos

Os atributos armazenam as características e informações associadas a cada objeto.

```
public class Carro {

    String marca;
    String modelo;
    String cor;
    int ano;
    double velocidade;

}
```

Embora todos os objetos `Carro` tenham esses atributos, os valores armazenados podem ser diferentes em cada instância.

10. DADOS EM MEMÓRIA

O estado de um objeto

```
Carro carro = new Carro();

carro.marca = "Toyota";
carro.modelo = "Corolla";
carro.velocidade = 0;
```

> O conjunto dos valores atuais dos atributos forma o **estado do objeto**.

```
carro.velocidade = 80; // o estado foi alterado
```

11. COMPORTAMENTOS

Métodos

Se os atributos representam o que um objeto **possui**, os métodos representam aquilo que ele **faz**.

```
public class Carro {

    double velocidade;

    void acelerar() {
        velocidade = velocidade + 10;
    }

}
```

> O método `acelerar()` modifica o próprio estado do objeto.

12. COMUNICAÇÃO COM OBJETOS

Chamando um método

```
Carro carro = new Carro();

carro.acelerar();
carro.acelerar();
```

Se cada chamada aumentar a velocidade em 10 km/h, após duas chamadas o objeto possuirá velocidade igual a 20.

> **Operador ponto:**
>
> `objeto.atributo` acessa um dado e `objeto.metodo()` solicita a execução de um comportamento.

13. EXEMPLO COMPLETO

Classe Carro

```
public class Carro {

    String marca;
    String modelo;
    int ano;
    double velocidade;

    void acelerar() {
        velocidade = velocidade + 10;
    }

    void frear() {
        velocidade = velocidade - 10;
    }

    void mostrarDados() {
        System.out.println("Marca: " + marca);
        System.out.println("Modelo: " + modelo);
        System.out.println("Ano: " + ano);
        System.out.println("Velocidade: " + velocidade);
    }
}
```

14. EXECUÇÃO

Utilizando a classe no método main

```
public class Principal {

    public static void main(String[] args) {

        Carro carro = new Carro();

        carro.marca = "Toyota";
        carro.modelo = "Corolla";
        carro.ano = 2025;

        carro.mostrarDados();
        carro.acelerar();
        carro.acelerar();
        carro.mostrarDados();
    }
}
```

15. MÉTODOS MAIS FLEXÍVEIS

Métodos com parâmetros

Um método pode receber informações no momento em que é executado.

```
void acelerar(double valor) {
    velocidade = velocidade + valor;
}
```

```
carro.acelerar(20);
carro.acelerar(50);
```

O parâmetro `valor` permite reutilizar o mesmo método para diferentes quantidades.

16. RESULTADOS

Métodos com retorno

```
double obterVelocidade() {
    return velocidade;
}
```

O tipo `double` antes do nome do método indica que um valor desse tipo será devolvido.

```
double velocidadeAtual = carro.obterVelocidade();

System.out.println(velocidadeAtual);
```

17. APLICAÇÃO

Exemplo: classe Aluno

```
public class Aluno {

    String nome;
    double nota1;
    double nota2;

    double calcularMedia() {
        return (nota1 + nota2) / 2;
    }
}
```

```
Aluno aluno = new Aluno();

aluno.nome = "Maria";
aluno.nota1 = 8.0;
aluno.nota2 = 7.0;

System.out.println(aluno.calcularMedia());
```

18. INICIALIZAÇÃO

Construtores

O construtor é executado no momento da criação do objeto e pode ser utilizado para garantir que a instância já seja criada com informações necessárias.

```
public class Aluno {

    String nome;
    int idade;

    public Aluno(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}
```

```
Aluno aluno = new Aluno("Carlos", 20);
```

19. REFERÊNCIA AO PRÓPRIO OBJETO

A palavra-chave `this`

```
public Aluno(String nome) {
    this.nome = nome;
}
```

this.nome

Refere-se ao atributo `nome` pertencente ao objeto atual.

nome

Refere-se ao parâmetro recebido pelo construtor.

20. EXEMPLO INTEGRADO

Classe Produto

```
public class Produto {

    String nome;
    double preco;
    int quantidade;

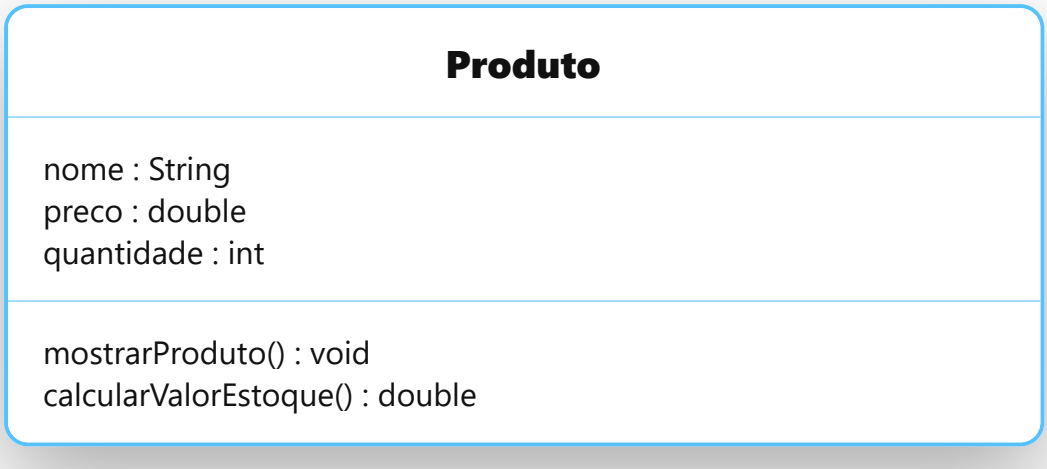
    public Produto(String nome, double preco, int quantidade)
{
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    double calcularValorEstoque() {
        return preco * quantidade;
    }

    void mostrarProduto() {
        System.out.println("Produto: " + nome);
        System.out.println("Preço: R$ " + preco);
        System.out.println("Quantidade: " + quantidade);
    }
}
```

21. REPRESENTAÇÃO VISUAL

Representação conceitual de uma classe



Essa forma de representação será retomada posteriormente durante o estudo de **UML e diagramas de classes**.

22. PROTEÇÃO DOS DADOS

Introdução ao encapsulamento

Até aqui os atributos foram acessados diretamente. Em aplicações orientadas a objetos, é comum restringir esse acesso utilizando `private`.

```
public class Produto {

    private String nome;
    private double preco;

}
```

> **Encapsulamento** permite que a própria classe controle como seus dados podem ser consultados ou modificados.

23. ACESSO CONTROLADO

Getters e Setters

```
public class Produto {

    private String nome;

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

```
produto.setNome("Notebook");
System.out.println(produto.getNome());
```

24. REGRA DE NEGÓCIO

Por que controlar o acesso?

O acesso controlado permite impedir estados inválidos.

> Um preço negativo, por exemplo, normalmente não representa um estado válido para um produto.

```
public void setPreco(double preco) {

    if (preco ⩾ 0) {
        this.preco = preco;
    }
}
```

A classe passa a ser responsável por proteger a consistência dos próprios dados.

25. VISÃO GERAL

Os quatro pilares da POO

Abstração

Selecionar e representar somente as características relevantes para o problema.

Encapsulamento

Proteger dados e definir formas controladas de acesso ao estado interno.

Herança

Permitir que uma classe aproveite características e comportamentos definidos em outra.

Polimorfismo

Permitir diferentes comportamentos por meio de uma mesma estrutura de referência.

> Nesta aula, o foco principal permanece em classe, objeto, atributo, método, construtor e uma introdução ao encapsulamento.

26. EXERCÍCIO GUIADO

Conta Bancária

Criar a classe **ContaBancaria** com os atributos `titular`, `numero` e `saldo`.

Implementar os métodos `depositar()`, `sacar()` e `mostrarSaldo()`.

```
public class ContaBancaria {

    String titular;
    int numero;
    double saldo;

    void depositar(double valor) {
        saldo += valor;
    }

    void sacar(double valor) {
        saldo -= valor;
    }

    void mostrarSaldo() {
        System.out.println("Saldo: R$ " + saldo);
    }
}
```

27. TESTE DO OBJETO

Instanciando e utilizando a conta

```
ContaBancaria conta = new ContaBancaria();

conta.titular = "João";
conta.numero = 1234;

conta.depositar(1000);
conta.sacar(250);
conta.mostrarSaldo();
```

> Resultado esperado: **Saldo: R$ 750.0**

28. PENSAMENTO ORIENTADO A OBJETOS

Existe um problema nesse saque?

```
void sacar(double valor) {
    saldo -= valor;
}
```

O código permite sacar qualquer valor, mesmo quando o saldo é insuficiente.

```
void sacar(double valor) {

    if (valor ⩽ saldo) {
        saldo -= valor;
    } else {
        System.out.println("Saldo insuficiente.");
    }
}
```

> Um objeto não precisa apenas guardar dados. Ele também pode garantir as **regras de negócio** relacionadas a esses dados.

29. ATIVIDADE PRÁTICA

Sistema de Cadastro de Livros

Proposta

Desenvolver uma aplicação Java simples aplicando os conceitos apresentados nesta aula.

Classe: `Livro`

Atributos: título, autor, ano e preço.

Construtor: deverá receber os dados necessários para criar um livro.

Métodos: `mostrarDados()` e `aplicarDesconto(double percentual)`.

30. REQUISITOS

Como a atividade deverá funcionar

```
Livro livro1 = new Livro(
    "Programação Java",
    "João Silva",
    2026,
    120
);

livro1.mostrarDados();
livro1.aplicarDesconto(10);
livro1.mostrarDados();
```

- Criar pelo menos **três objetos** da classe `Livro`.
- Utilizar valores diferentes em cada objeto.
- Demonstrar a alteração do preço após a aplicação do desconto.

31. DESAFIO ADICIONAL

Evolução da atividade

1. Encapsular

Alterar todos os atributos para `private`.

2. Criar acesso

Adicionar getters e setters quando necessários.

3. Validar

Impedir que o preço seja definido com valor negativo.

4. Testar

Criar situações válidas e inválidas para verificar o comportamento da classe.

32. SÍNTESE DA AULA

Da classe ao comportamento

CLASSE → **OBJETOS** → **ESTADO** +

COMPORTAMENTO

Exemplo: Aluno

Atributos: nome, nota1 e nota2.

Métodos: calcularMedia() e mostrarDados().

Diversos objetos podem ser criados a partir da mesma classe, cada um com seu próprio estado.

33. CONTINUIDADE

Sequência de estudo

Aula 02

Encapsulamento

Modificadores de acesso, getters, setters, validações e construtores.

Aula 03

Relacionamentos

Associação e comunicação entre objetos.

Aula 04

Herança

Generalização, especialização e reutilização.

Aula 05

Polimorfismo

Sobrescrita e diferentes implementações para uma mesma abstração.

Esses fundamentos serão utilizados posteriormente na construção de aplicações Java organizadas em camadas, incluindo padrões como MVC e DAO.